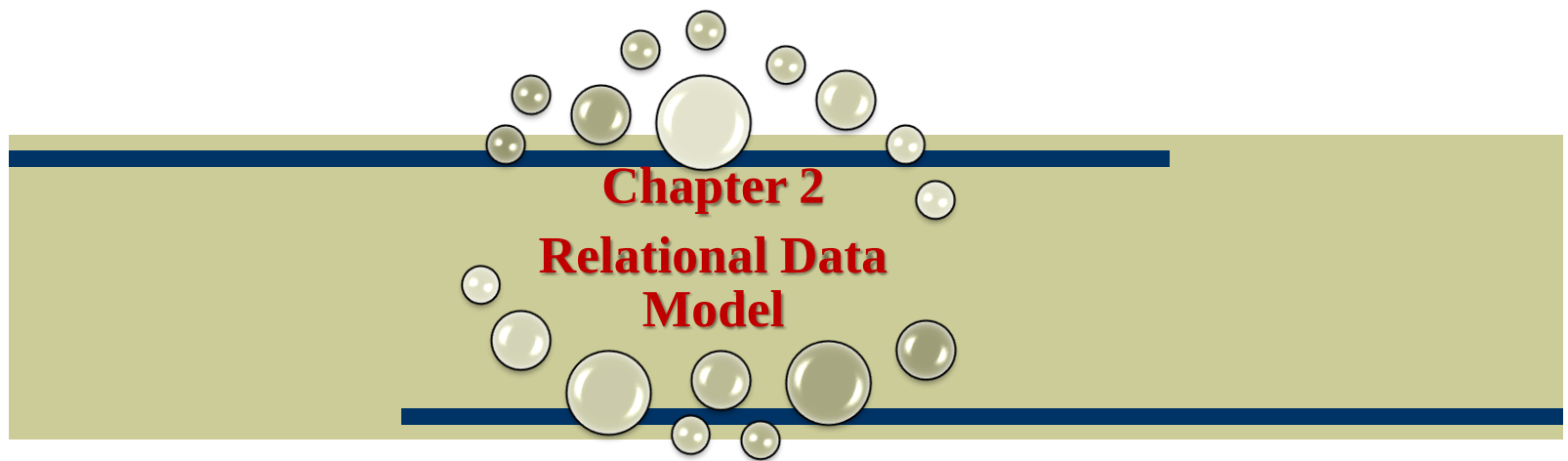




2.5 DDL Commands: **CREATE, ALTER, DROP, TRUNCATE, DESC,** **RENAME.**

	Prepared By
	Mohammad Ali
	Lecturer (Selection Grade)
	M. H. Saboo Siddik Polytechnic, Mumbai

SQL as Data Definition Language

- ◆ DDL is used to define the **schema** of the database (a collection of database objects, e.g. tables, views, indexes, sequences).
 - Create, Drop, Alter or Rename a *table*
 - Create or Drop an *Index*
 - Define Integrity constraints

Create Command -Table Creation

- ◆ One of the first steps in creating a database is to create the tables that will store company / organization's data.
- ◆ In order to create a table , four pieces of information must be determined:
 1. **The table name**
 2. **The column (field) names**
 3. **Column data types and**
 4. **Column sizes**

Naming Table and Columns

- ◆ Table and Column names should be meaningful and reflect the nature of the data that is to be stored.
- ◆ If the data stored is about the products that a firm sells , then the table should probably be named *product!*
- ◆ If products are identified by a string of characters, then the column that stores the product information data should be named *product_number*, or *product_code*.

Picking a data type

- ◆ The data type chosen for a column determines the nature of the data that can be stored in the column.
- ◆ This is termed the ***Domain*** of valid column values.
- ◆ Oracle provides 14 pre-determined data types as well as the ability to declare user defined data types.

CREATE TABLE - SYNTAX:

```
CREATE TABLE <table_name>  
( column1 datatype(size),  
  column2 datatype(size),  
  column3 datatype(size),  
  ....  
  columnN datatype(size));
```

Restrictions for creating a table:

- ◆ Table names and column names **must begin with a letter**.
- ◆ Table names and column names can be **1 to 30** characters long.
- ◆ Table names must contain only the characters **A-Z, a-z, 0-9, _, \$ and #**.
- ◆ Table name **should not be same as the name of another database object**.
- ◆ Table name **must not be** an **ORACLE reserved word**.
- ◆ **Column** names should not be **duplicate** within a table definition.

Create Table: Example

CREATE TABLE employee

```
( emp_number char(4),  
  fname VARCHAR2(8),  
  lname VARCHAR2(9),  
  adharno VARCHAR2(15) NOT NULL,  
  bdate DATE,  
  address VARCHAR2(27),  
  sex VARCHAR2(1),  
  salary NUMBER(7) NOT NULL,  
  dno NUMBER(2) NOT NULL,  
  PRIMARY KEY (emp_number)) ;
```

CREATE TABLE dependant

```
( lname d_last_name NOT NULL,  
  fname VARCHAR(18) NOT NULL,  
  adharno d_adhrano NOT NULL,  
  date_of_birth DATE,  
  employee_adharno d_adhrano NOT NULL );
```

- ♦ Creating a simple table that stores ten items of information about **employees** for an organization.
- ♦ The table is named “**employee**” and stores information about each employee’s aadhar number, last name, first name, date hired, and salary etc.

The **ALTER TABLE** Command

ALTER TABLE statement is used to:

- Add a new attribute
- Modify an existing attribute
- Define a default value for the new attribute
- Drop an attribute
- ◆ If a column modification will result in a column that is smaller than was originally specified, Oracle will return an error message if data rows exist such that their data will not fit into the new specified column size.

The **ALTER TABLE** Command

Use the ALTER TABLE statement to add, modify, or drop columns.

```
ALTER TABLE <table_name>
ADD    (column1 datatype(size), column2 datatype(size)
        ...
        columnn datatype(size));
```

```
ALTER TABLE <table_name>
MODIFY  (column1 datatype(size), column2 datatype(size)
        ...
        columnn datatype(size));
```

```
ALTER TABLE table
DROP      (column);
```

Adding a Column

- We use the ADD clause to add columns.

```
ALTER TABLE dept80  
ADD (job_id VARCHAR2(9));  
Table altered.
```

EMPLOYEE_ID	LAST_NAME	ANNSAL	HIRE_DATE	JOB_ID
145	Russell	14000	01-OCT-96	
146	Partners	13500	05-JAN-97	
147	Errazuriz	12000	10-MAR-97	
148	Cambrault	11000	15-OCT-99	
149	Zlotkey	10500	29-JAN-00	

...

Modifying a Column

- We can change a column's data type, size, and default value.

```
ALTER TABLE dept80  
MODIFY (last_name VARCHAR2(30));  
Table altered.
```

Dropping a Column

Use the DROP COLUMN clause to drop columns you no longer need from the table.

```
ALTER TABLE dept80  
DROP COLUMN job_id;  
Table altered.
```

EMPLOYEE_ID	LAST_NAME	ANNSAL	HIRE_DATE
145	Russell	14000	01-OCT-96
146	Partners	13500	05-JAN-97
147	Errazuriz	12000	10-MAR-97
148	Cambrault	11000	15-OCT-99
149	Zlotkey	10500	29-JAN-00

The SET UNUSED Option

- We use the SET UNUSED option to mark one or more columns as unused.
- We use the DROP UNUSED COLUMNS option to remove the columns that are marked as unused.

```
ALTER TABLE <table_name>  
SET UNUSED(<column_name>);
```

OR

```
ALTER TABLE <table_name>  
SET UNUSED COLUMN <column_name>;
```

```
ALTER TABLE <table_name>  
DROP UNUSED COLUMNS;
```

Another Example

- creating student table

```
CREATE TABLE student (studentid NUMBER(5,0),  
    first_name VARCHAR2(25),  
    last_name Varchar2(10),  
    dept VARCHAR2(15),  
    gpa NUMBER(6,3) default 4.0,  
    tutorid NUMBER(5,0),  
    home_phone varchar2(10));
```

Changing Table Components with ALTER

- Changing Attributes:

```
ALTER TABLE student MODIFY last_name  
VARCHAR(35);
```

- alter table student modify gpa
NUMBER(6,3) default 0.0;

- Adding Attributes:

```
ALTER TABLE student ADD admission DATE;
```

- Removing Attributes (not widely implemented):

```
ALTER TABLE student DROP column  
home_phone
```


Adding a Constraint Syntax

Use the ALTER TABLE statement to:

- Add or drop a constraint, but not modify its structure
- Enable or disable constraints
- Add a NOT NULL constraint by using the MODIFY clause

```
ALTER TABLE <table_name>  
ADD [CONSTRAINT <constraint_name>]  
type (<column_name>);
```

Adding a Constraint

Add a FOREIGN KEY constraint to the EMP2 table indicating that a manager must already exist as a valid employee in the EMP2 table.

```
ALTER TABLE emp2  
modify employee_id Primary Key;  
Table altered.
```

```
ALTER TABLE emp2  
ADD CONSTRAINT emp_mgr_fk  
    FOREIGN KEY(manager_id)  
    REFERENCES emp2(employee_id);  
Table altered.
```

ON DELETE CASCADE

Delete child rows when a parent key is deleted.

```
ALTER TABLE Emp2 ADD CONSTRAINT emp_dt_fk  
FOREIGN KEY (Department_id)  
REFERENCES departments ON DELETE CASCADE);  
Table altered.
```

Dropping a Constraint

- Remove the manager constraint from the EMP2 table.

```
ALTER TABLE emp2  
DROP CONSTRAINT emp_mgr_fk;  
Table altered.
```

- Remove the PRIMARY KEY constraint on the DEPT2 table and drop the associated FOREIGN KEY constraint on the EMP2 . DEPARTMENT_ID column.

```
ALTER TABLE dept2  
DROP PRIMARY KEY CASCADE;  
Table altered.
```

Disabling Constraints

- Execute the DISABLE clause of the ALTER TABLE statement to deactivate an integrity constraint.
- Apply the CASCADE option to disable dependent integrity constraints.

```
ALTER TABLE emp2  
DISABLE CONSTRAINT emp_dt_fk;  
Table altered.
```

Enabling Constraints

- Activate an integrity constraint currently disabled in the table definition by using the `ENABLE` clause.

```
ALTER TABLE          emp2  
ENABLE CONSTRAINT emp_dt_fk;  
Table altered.
```

- A `UNIQUE` index is automatically created if you enable a `UNIQUE` key or `PRIMARY KEY` constraint.

Cascading Constraints

- The `CASCADE CONSTRAINTS` clause is used along with the `DROP COLUMN` clause.
- The `CASCADE CONSTRAINTS` clause drops all referential integrity constraints that refer to the primary and unique keys defined on the dropped columns.
- The `CASCADE CONSTRAINTS` clause also drops all multicolumn constraints defined on the dropped columns.

Cascading Constraints

Example:

```
ALTER TABLE emp2  
DROP COLUMN employee_id CASCADE CONSTRAINTS;  
Table altered.
```

```
ALTER TABLE test1  
DROP (pk, fk, col1) CASCADE CONSTRAINTS;  
Table altered.
```


DROP TABLE Command

- ◆ The drop table command removes the data as well as definition(structure) of an Oracle Table.
- ◆ When we drop a table, the database loses structure with all the data in the table, related constraints and all the indexes associated with it.
- ◆ Syntax:

Drop Table <table_name>

- ◆ Example:

Drop Table dept80;

Removing Schema Components with DROP

```
DROP TABLE table_name
```

```
DROP TABLE table_name CASCADE
```

```
DROP TABLE table_name RESTRICT
```

```
DROP INDEX index_name
```

```
DROP CONSTRAINT  
table_name.constraint_name
```

TRUNCATE TABLE Command:

- ◆ The truncate table command is used to remove all rows from a table and to release the storage space used by the table keeping the table definition(structure) intact.

- ◆ Syntax:

Truncate Table <table_name>

- ◆ Example:

Truncate Table dept80;

Viewing a Table Description

- ◆ The **DESCRIBE (DESC)** command can display the column names and data types for any table.
- ◆ This command can be used when exact data types and column sizes for a table are unknown.

Describe Command

```
DESC employee;
```

Name	Null?	Type
-----	-----	-----
EMP_SSN	NOT NULL	CHAR(9)
EMP_LAST_NAME	NOT NULL	VARCHAR2(25)
EMP_FIRST_NAME	NOT NULL	VARCHAR2(25)
EMP_DATE_OF_BIRTH		DATE
EMP_SALARY	NOT NULL	NUMBER(7,2)
EMP_PARKING_SPACE		NUMBER(4)

- ◆ Note that while *emp_ssn* column was specified to have a PRIMARY KEY constraint, the *Null?* Column displayed in the table description indicates whether or not a column is constrained as NOT NULL.

Renaming a Table

- ◆ A table can be renamed with the **RENAME** command.
- ◆ This command does not affect table structure or data; it simply gives the current table a new name.

- ◆ Syntax:

Rename <table_name> To <New_table_name>

- ◆ Example:

RENAME employee TO worker;